UF1 – PROGRAMACIÓ ESTRUCTURADA

ELEMENTS D'UN PROGRAMA DADES I OPERADORS

Una dada sempre té associada un tipus de dada, que determina el conjunt de valors que pot prendre.

Números enters

Els números **enters** són aquells que no tenen decimals, tant positius com negatius (a més del zero). En Python es poden representar mitjançant el tipus int (de integer, sencer) o el tipus long (llarg). L'única diferència és que el tipus long permet emmagatzemar nombres més grans. És aconsellable no utilitzar el tipus long llevat que sigui necessari, per no malbaratar memòria.

En python 2.x exisita el tipus **int** que s'implementava a baix nivell mitjançant un tipus **long** de C. I com que Python utilitza C per sota, igual que C, i a diferència de Java, el rang dels valors que pot representar depèn de la plataforma.

En plataformes de 64 bits, el rang és de -9.223.372.036.854.775.808 fins 9.223.372.036.854.775.807.

En python3 en assignar un nombre a una variable aquesta passarà a tenir tipus **long** de C.

El tipus **int** de Python permet emmagatzemar números de qualsevol precisió, limitat per la memòria disponible a la màquina.

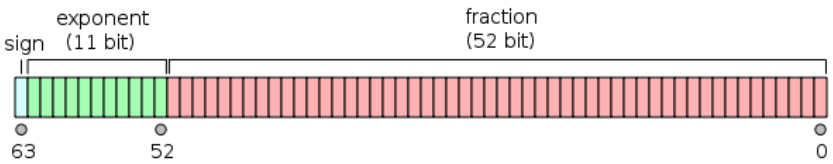
```
enter=23
```

Números reals

Els números reals són els que tenen decimals. En Python s'expressen mitjançant el tipus float. En altres llenguatges de programació, com C, tenim també el tipus double, similar a float però de major precisió (double = doble precisió). Python, però, implementa el seu tipus float a baix nivell mitjançant una variable de tipus **double de C**, és a dir, utilitzant 64 bits, després en Python sempre s'utilitza doble precisió.

S'utilitzarà 1 bit per al signe (positiu o negatiu), 11 bits per l'exponent i 52 bits per la mantissa.

```
real = 0.2703
```



OPERADORS ARITMÈTICS

Operador	Descripción	Ejemplo
+	Suma	r=3+2 # r es 5
-	Resta	r=4-7 # r es -3
-	Negació	r=-7 # r es -7
*	Multiplicació	r=2*6 # r es 12
**	Exponent	r=2**6 # r es 64
/	Divisió	r=3.5/2 # r es 1.75
//	Divisió Entera	r=3.5//2 r es 1.0
%	Mòdul	r=7%2 # r es 1

Números complexes

Els nombres complexos són aquells que tenen part imaginària. S'emmagatzema usant **coma flotant**, pel fet que aquests números són una extensió dels nombres reals. En concret s'emmagatzema en una estructura de C, composta per **dues variables de tipus double**, servint una d'elles per emmagatzemar la part real i l'altra per a la part imaginària.

```
complexe = 2.1 + 7.8j
```

Binari, octal i hexadecimal

Tant a python2.x com a python3.x podrem escriure números en binari (base 2), octal (base 8) i en hexadecimal (base 16).

Un número prefixat per 0b (zero i "b" minúscula o majúscula "B") s'interpretarà com un número binari:

```
binari_number = 0b1101
```

Un número prefixat per 0o (zero i "o" minúscula o majúscula "O") s'interpretarà com un número octal:

```
octal_number = 0o10
```

Un número prefixat per 0x (zero i "x" minúscula o majúscula "X") s'interpretarà com un número hexadecimal:

```
hex_number = 0xA0F
```

Podem transformar decimal a hexadecimal, octal i binari de la següent manera:

```
>>> hex(10)
'0xa'
```

```
>>> oct(8)
'0o10'
```

```
>>> bin(8)
'0b1000'
```

Podem transformar hexadecimal, octal o binari a decimal de la següent manera:

```
>>> int(0xb)
11
```

```
>>> int(0o10)
8
```

```
>>> int (0b1010)
10
```

El booleà

El tipus booleà només pot tenir dos valors: True (cert) i False (fals)

```
aT = True
```

OPERADORS RELACIONALS

Operador	Descripció	Exemple
==	Són iguals a i b?	r = 5 == 3 # r és False
!=	Són diferents a i b?	r = 5 != 3 # r és True
<	És a menor que b?	r = 5 < 3 # r és False
>	És a major que b?	r = 5 > 3 # r és True
<=	És a menor o igual que b?	r = 5 <= 5 # r és True
>=	És a major o igual que b?	r = 5 >= 3 # r és True

Cadenes

Les cadenes no són més que **text** tancat entre cometes simples ('cadena') o dobles ("cadena"). Dins de les cometes es poden afegir caràcters especials escapant-los amb '\', com '\n', el caràcter de nova línia, o '\t', el de tabulació.

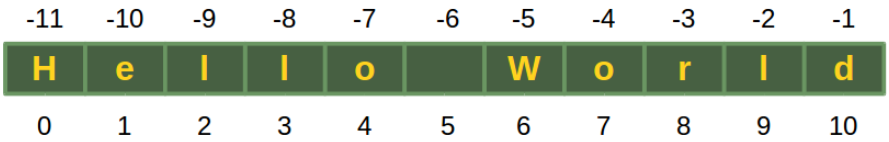
En python 2.X una cadena pot estar precedida pel caràcter 'u' o el caràcter 'r', els quals indiquen, respectivament, que es tracta d'una cadena que utilitza codificació Unicode i una cadena raw (de l'anglès, crua). Les cadenes raw es distingeixen de les normals en què els caràcters escapats mitjançant la barra invertida (\) no se substitueixen per les seves contrapartides. Això és especialment útil, per exemple, per a les expressions regulars.

```
raw = r"\n"
```

També és possible tancar una cadena entre triples cometes (simples o dobles). D'aquesta manera podrem escriure el text en diverses línies, i en imprimir la cadena, es respectaran els salts de línia que vam introduir sense haver de recórrer al caràcter \n, així com les cometes sense haver de escapar-les.

En **python 3.x** totes les cadenes són **unicode.**

Un string en python consisteix en una seqüencia de caràcters – lletres, números o caràcter especials – indexats.




```
cadena='Hello World'                    print( cadena[10] )   # d
print( cadena[0] )    # H               print( cadena[-1] )   # d
print( cadena[1] )    # e               print( cadena[0] )   # H
print( cadena[2] )    # l               print( cadena[-11] ) # H

# SUBCADENES (sub-Strings)
print( cadena[:5] )   # Hello           print( cadena[6:] )   # World
print( cadena[0:5] )  # Hello           print( cadena[6:11] ) # World
```

OPERACIONS AMB CADENES

Sumar cadenes entre si (concatenar).

```
a = "uno"

b = "dos"

c = a + b # c es "unodos"
```

Multiplicar una cadena s per un nombre k (i el resultat és la concatenació de s amb si mateixa, k vegades):

```
a = "uno"

b = "dos"

c = a * 3 # c es "unounouno"
```

Longitud d'una cadena: Podem esbrinar la longitud d'una cadena utilitzant una funció proveïda per Python: len {}.

```
len("programas ")

10
```

Hi ha una cadena especial, que anomenarem cadena buida, que és la cadena que no conté cap caràcter (es la indica només amb un apòstrof o cometa que obre, i un apòstrof o cometa que tanca), i que per tant té longitud zero:

```
a=""

len(a)

retorna 0
```

Accedir a la posició d'una cadena. Si volem esbrinar quin és el caràcter que està en la posició i-èsima d'una cadena, escriurem a [i] per parlar de la posició i-èsima de la cadena a.

```
a="Veronica"

a[1]

retorna e
```

Podrem accedir a la darrera lletra de la cadena s amb:

```
s[len(s)-1]

o amb

s[-1]
```

SEQÜENCIES D'ESCAPAMENT A LES CADENES

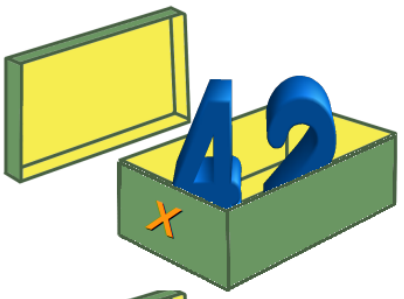
El caràcter de barra invertida (\) s'utilitza per escapar dels caràcters, és a dir, per "escapar" del significat especial, que d'aquesta manera tindria aquest caràcter.

Seqüència d'escape	Significat
\\	Barra invertida (\)
\'	Cometa simple (')
\"	Cometa doble (")
\b	Retrocés ASCII (BS)
\f	Avançar pàgina (FF) ASCII FormFeed
\n	Avançar línia (LF) ASCII LineFeed
\r	Retorn de carro (CR) ASCII Carriage Return
\t	Tabulació horitzontal (TAB)
\v	Tabulació vertical (VT)

VARIABLES

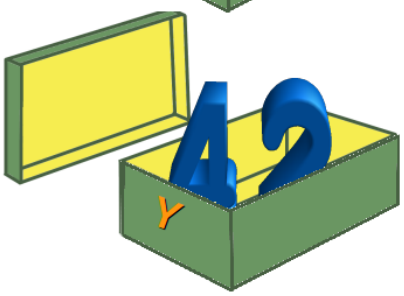
Una variable és una cosa que pot canviar. Una variable és una manera de referir-se a una ubicació de memòria usada per un programa informàtic. En molts llenguatges de programació, una variable és un **nom simbòlic** per a aquesta **ubicació física**. Aquesta ubicació de la memòria conté valors, com ara números, text o tipus més complicats. Podem utilitzar aquesta variable per dir-li a l'ordinador que guardi algunes dades en aquesta ubicació o per recuperar algunes dades d'aquesta ubicació.

Una variable es pot veure com **un contenidor** (o alguns diuen *pigeonhole*) per emmagatzemar determinats valors. Mentre s'està executant el programa, les variables poden ser accedides i, de vegades, es modifiquen, és a dir, un nou valor s'assignarà a una variable.



Posar valors a les variables es pot realitzar amb assignacions. La manera d'assignar valors a les variables és gairebé igual a tots els llenguatges de programació. En la majoria dels casos s'utilitza el signe "=" igual. El valor es posa al costat dret i el nom de la variable al costat esquerre.

```
x = 42;
y = 42;
```



Hem assignat el valor 42 a ambdues variables i podem veure que dos Números es guarden físicament a la memòria, que corresponen a les dues caixes de sabates de la imatge del costat.

NOMS DE VARIABLES VÀLIDS

Un identificador de Python és un nom que s'utilitza per identificar una variable, funció, classe, mòdul o un altre objecte.

Un nom de variable i un identificador poden consistir en lletres **majúscules [A-Z]**, les **minúscules [a-z]**, el **guió baix** _ i, excepte el primer caràcter, els **dígits** del 0 al 9. Python 3.X està basat en Unicode. Això significa que els noms de variables i els noms d'identificadors també poden contenir caràcters Unicode.

Els identificadors tenen una longitud il·limitada.

El fet que els noms d'identificador distingeixin majúscules i minúscules pot causar problemes a alguns usuaris de Windows, on els noms dels fitxers no són sensibles a majúscules i minúscules, per exemple.

Les excepcions de les regles anteriors són les paraules clau especials de Python que no es poden fer servir com a identificadors:

```
and, as, assert, break, class, continue, def, del, elif, else,
except, False, finally, for, from, global, if, import, in, is,
lambda, None, nonlocal, not, or, pass, raise, return, True, try,
while, with, yield
```

CONVENCIONS PER ALS NOMS D'IDENTIFICADORS

Si necessitem noms que consten de més d'una paraula, per exemple, el nom "maximum_height". Farem servir **el subratllat** com a separador de paraules, ja que no es permeten espais en blanc en noms de variables.

Algunes persones prefereixen escriure noms de variables a l'anomenada notació de CamelCase, on cada paraula comença amb una majúscula, exemple, MinimumHeight.

Hi ha una "guerra" permanent entre els seguidors del CamelCase i els amants del subratllat.

La Guia d'estil per al codi Python recomana la notació de subratllat per als noms de les variables i els noms de les funcions.

Alguns noms han de ser evitats per als noms de les variables: **no feu servir els caràcters 'l' (minúscula "L"), "O" , o "I"** com noms de variable de caràcter únic. S'ha d'evitar, perquè aquests caràcters no es distingeixen dels nombres 1 i zero en algunes fonts.

La guia d'estil python diu:
"Tots els identificadors de la biblioteca estàndard de Python HAN D'utilitzar només identificadors ASCII , i HAURIEN DE fer servir paraules en anglès sempre que sigui possible. A més, els literals de cadena i els comentaris també han de ser a ASCII. [..]"

EXEMPLES DE NOMS DE VARIABLES

```
PreuCompra = 10.3
preu_compra = 10.3

strDNI = '45621187F'
str_DNI = '45621187F'
int_DNI = 45621187

velocitat = 100
velocitatTren = 230

iva21 = 21 / 100
```

Evitar noms de variable amb ...:
- accents, espais, o caràcters especials
- variables només amb una lletra (a, b, ...)
- sense significat (hla, aa, a1, a2, ...)

ENTRADA DE TECLAT (LA FUNCIÓ INPUT)

No hi ha molts programes sense cap entrada. L'entrada pot venir d'una base de dades, una altra computadora, clics i moviments del ratolí o des d'Internet. Tot i així, en la majoria dels casos, l'entrada prové **del teclat**. A aquest efecte, Python proporciona l'entrada de la funció **input()**.

Si es crida a la funció **input()**, el flux del programa es detindrà fins que l'usuari hagi donat una entrada i hagi finalitzat l'entrada amb la tecla de retorn. El text del paràmetre és opcional i serà imprès a la pantalla.

L'entrada de l'usuari serà retornada com una cadena. Si aquesta cadena s'ha de transformar en un altre tipus de dada, podem utilitzar una funció de càsting o la funció eval.

```
name = input("Quin és el teu nom? ")
print("Encantat de conèixer-te " + name + "!")
age = input("Quina edat tens? ")
print("Encara tens " + age + " anys, " + name + "!")
```

i la sortida seria:

```
$ python input_test.py
Quin és el teu nom?  "Frank"
Encantat de conèixer-te Frank!
Quina edat tens? 42
Encara tens 42 anys, Frank!
```

Podem fer servir la funció **type()** per avaluar el tipus de dada introduïda:

```
#exemple de type
age=input("Quina és la teva edat: ")
print (age, type(age))
age=int(input("Quina és la teva edat: "))
print (age, type(age))
age=float(input("Quina és la teva edat: "))
print (age, type(age))
```

SORTIDA PER PANTALLA (LA FUNCIÓ PRINT)

La sortida estàndard d'un programa és la pantalla, i python 2.x te una instrucció per a imprimir per pantalla que és print. No obstant això, python 3.x fa servir una funció també nomenada print, i com a funció anirà acompanyada de parèntesi i esperarà un o més arguments.

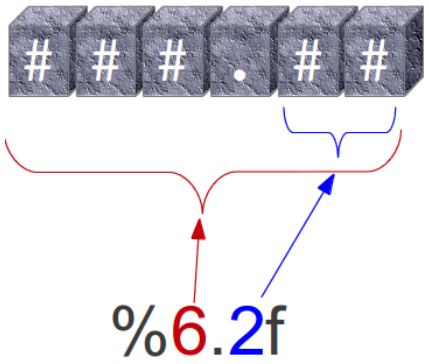
Aquesta funció pot imprimir números, cadenes, caràcters,...

També permet donar-li format a la sortida.

Heretat del python 2.6 tenim un print() amb format que s'està deixant d'utilitzar:

```
print ("%3d x %2d = %2d" % (num, a, num*a))
```

El "%6.2f" és una descripció del format d'un número flotant. S'introdueix amb el caràcter "%". A continuació, se segueix el nombre total de dígits que ha de contenir la cadena. Aquest número inclou el punt decimal i tots els dígits, és a dir abans i després del punt decimal. El número 59.058 ha de tenir format amb 8 caràcters. La part decimal del número ocupara 6 posicions i la part fraccionaria s'estableix en 2, és a dir, el número següent al "." en el nostre marcador de posició. Finalment, l'últim caràcter "f" del nostre marcador de posició és "float".



Conversió	Significat
d	Integer amb signe
i	Integer amb signe
o	Octal sense signe.
x	Hexadecimal sense signe..
X	Hexadecimal sense signe.
e	Floating point exponential (lowercase).
E	Floating point exponential (uppercase).
f	Floating point decimal.
F	Floating point decimal.
c	Caràcter simple.
r	String (cadena).
s	String (cadena).

Com que donar format no és la funció principal de la funció print(), li donarem pas a un mètode que donarà format abans d'imprimir. Aquest mètode es diu format. Un exemple seria:

```
>>> "Art: {a:5d},  Price: {p:8.2f}".format(a=453, p=59.058)

'Art:   453,  Price:    59.06'
```

La plantilla (o cadena de format) és una cadena que conté un o més codis de format (camps a substituir) incrustats. Els "camps a reemplaçar" estan envoltats per claudàtors {}. Els claudàtors i el "codi" de l'interior seran substituïts per un valor formatat per un dels arguments, d'acord amb les regles. Qualsevol altra cosa, que no estigui inclosa en claudàtors, s'imprimirà literalment, és a dir, sense canvis. Si s'ha d'imprimir un claudàtor, ha de ser escapat duplicant-lo: {{and}}.

Hi ha dos tipus d'arguments per al mètode .format (). La llista d'arguments comença amb zero o més arguments de posició (p0, p1, ...), pot ser seguit de zero o més arguments de paraules clau del nom del formulari name = value.

Es pot accedir a un paràmetre posicional del mètode de format posant l'índex del paràmetre després de l'obertura, {0} accedeix al primer paràmetre, {1} el segon i així successivament. L'índex pot estar seguit de dos punts i una cadena de format, que és similar a la notació del print() antic.

Ex. {0: 5d}
Si s'utilitzen els paràmetres de posició en l'ordre en què estan escrits, es poden ometre els especificadors de l'argument posicional a l'interior de les claus, de manera que '{} {} {}' correspon a '{0} {1} {2}'. Però són necessaris, si voleu accedir-hi en diferents comandes: '{2} {1} {0}'.

El següent diagrama amb un exemple d'ús mostra com funciona el mètode "format" de la cadena de treball per als paràmetres de posició:

```
print("First argument: {0}, second one: {1}".format(47,11))

#First argument: 47, second one: 11'

print("Second argument: {1}, first one: {0}".format(47,11))

#'Second argument: 11, first one: 47'

print("Second argument: {1:3d}, first one: {0:7.1f}".format(47.42,11))

#'Second argument:  11, first one:   47.42'

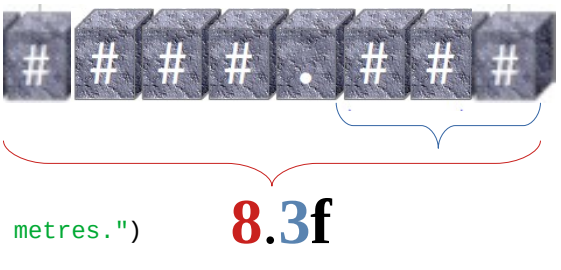
print("First argument: {}, second one: {}".format(47,11))

#'First argument: 47, second one: 11'

# arguments can be used more than once:

print( "various precisions: {0:6.2f} or {0:6.3f}".format(1.4148))

#'various precisions:   1.41 or  1.415'
```




```
#######################################################
### E X E M P L E S format a partir Python 3.6 ###
#######################################################

nom = "Juan"
edat = 20
alcada = 1.75
Print(f"Te llamas {nom}, tens {edat} anys i mesures {alcada} metres.")
# Te llamas Juan, tens 20 anys i mesures 1.75 metres.

print(f"Te llamas {nom}, tens {edat:4d} anys i mesures {alcada:8.3f} metres.")
# Te llamas Juan, tens   20 anys i mesures    1.750 metres.

# Si indiquem la quantitat de caràcters que volem imprimir com a mínim,
# aquest sistema per defecte utilitzarà una alineació esquerra, em canvi l'anterior amb
# "%" (o per defecte) dreta.
# Per canviar aquest comportament utilitzem els caràcters «<" (esquerra) i ">» (dreta).

for nom in ("Joan", "Pere", "Francesc"):
    print(f"{nom:>9}")
```

```
>>> a=453
>>> p=59.058
>>> print(f"Art: {a},  Price: {p}")
'Art: 453,  Price: 59.058'

>>> print(f"Art: {a:5d},  Price: {p:8.2f}")
'Art:   453,  Price:    59.06'

>>> print(f"Art: {a:10d},  Price: {p:.1f}")
'Art:        453,  Price: 59.1'
```

```
############################################################
### E X E M P L E S paràmetre END (a partir Python 3) ###
############################################################
https://www.tutorialstonight.com/python/print-in-python-3.php


print ("hello",end=" +") # will print hello +



# This Python program must be run with
# Python 3 as it won't work with 2.7.

# ends the output with a <space>
print("Welcome to" , end = ' ')
print("GeeksforGeeks", end = ' ')

Output :
      "Welcome to GeeksforGeeks "
```

Bibliografia i web-grafia

Teach your kids to Code, Bryson Payne, No starch Press

https://docs.python.org/3/tutorial/index.htm

https://www.python-course.eu/python3_variables.php

Python 3 al descubierto - 2a ed., Arturo Fernández, Alfaomega Grupo Editor, 30 de set. 2013

http://python-para-impacientes.blogspot.com.es/2014/01/entrada-estandar-input.html